



Should You Trust Your Copilot?

Limitations and merits of AI coding assistants

Emanuele Fabbiani
Head of AI @ xstream



1,000,000

Copilot users as of January 2023



“I use Copilot every day. [...] It doesn't save me a lot of thinking, but it helps me take care of the boring stuff. ”

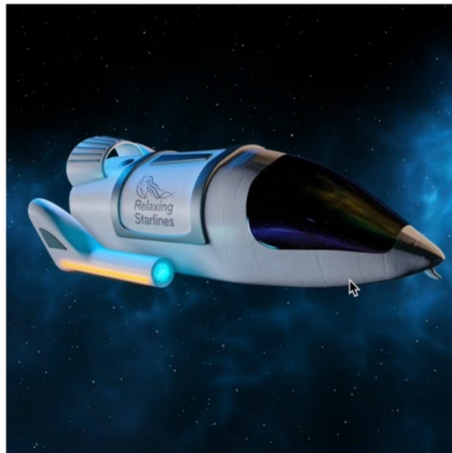
Guido Van Rossum, Inventor of Python



1. How does Copilot work?
2. What are the benefits of Copilot?
3. What are the problems with Copilot?



How does
Copilot work?

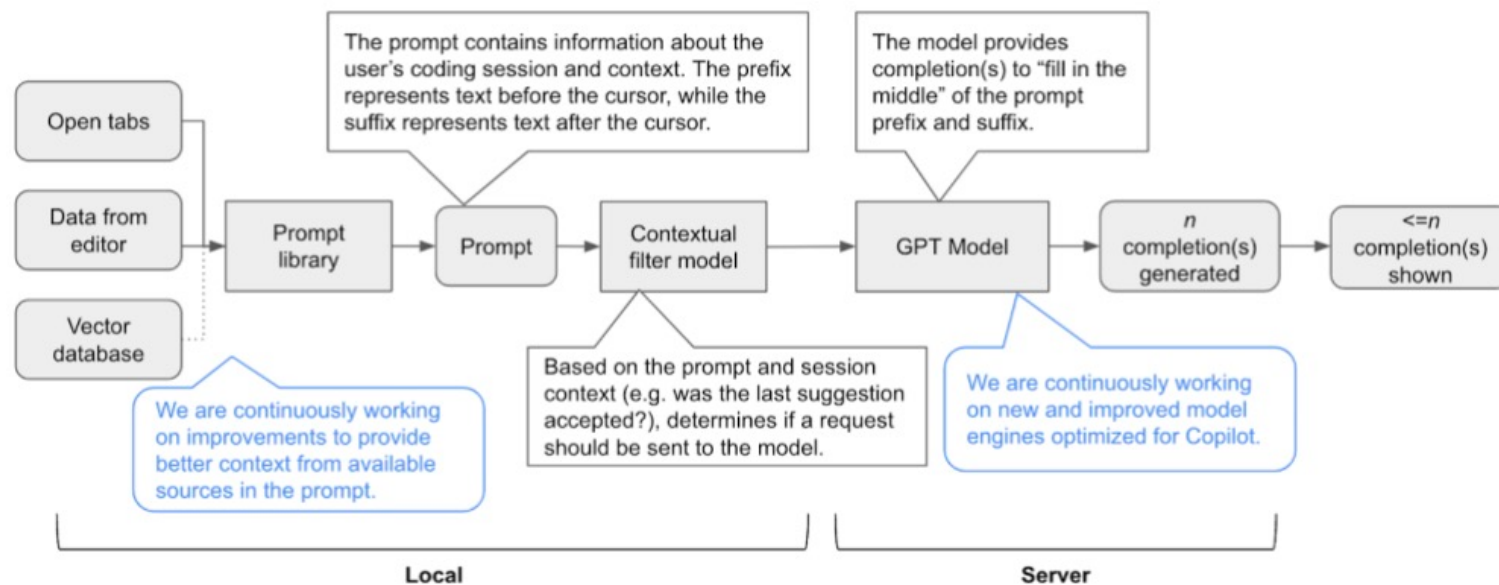


Add this image of a rocketship:
<https://i1.sndcdn.com/artworks-j8xjG7zc1wmTe07b-06l83w-t500x500.jpg>



```
/* Add this image of a
rocketship:
https://i1.sndcdn.com/artworks-
j8xjG7zc1wmTe07b-06l83w-
t500x500.jpg */
var rocketship =
document.createElement('img');
rocketship.src =
'https://i1.sndcdn.com/artwork
s-j8xjG7zc1wmTe07b-06l83w-
t500x500.jpg';
document.body.appendChild(rock
etship);
```





Simplified system diagram focused on model quality efforts. Made by Alice Li, machine learning researcher at GitHub.



What are the
benefits of
Copilot?



The numbers speak for themselves.

55%
faster coding

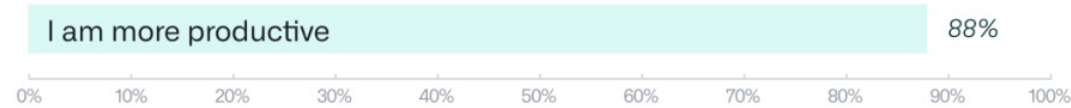
75%
more fulfilled

46%
code written

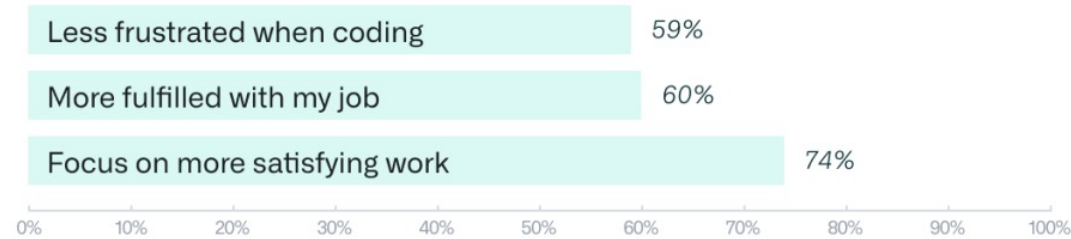


When using GitHub Copilot...

Perceived Productivity



Satisfaction and Well-being*

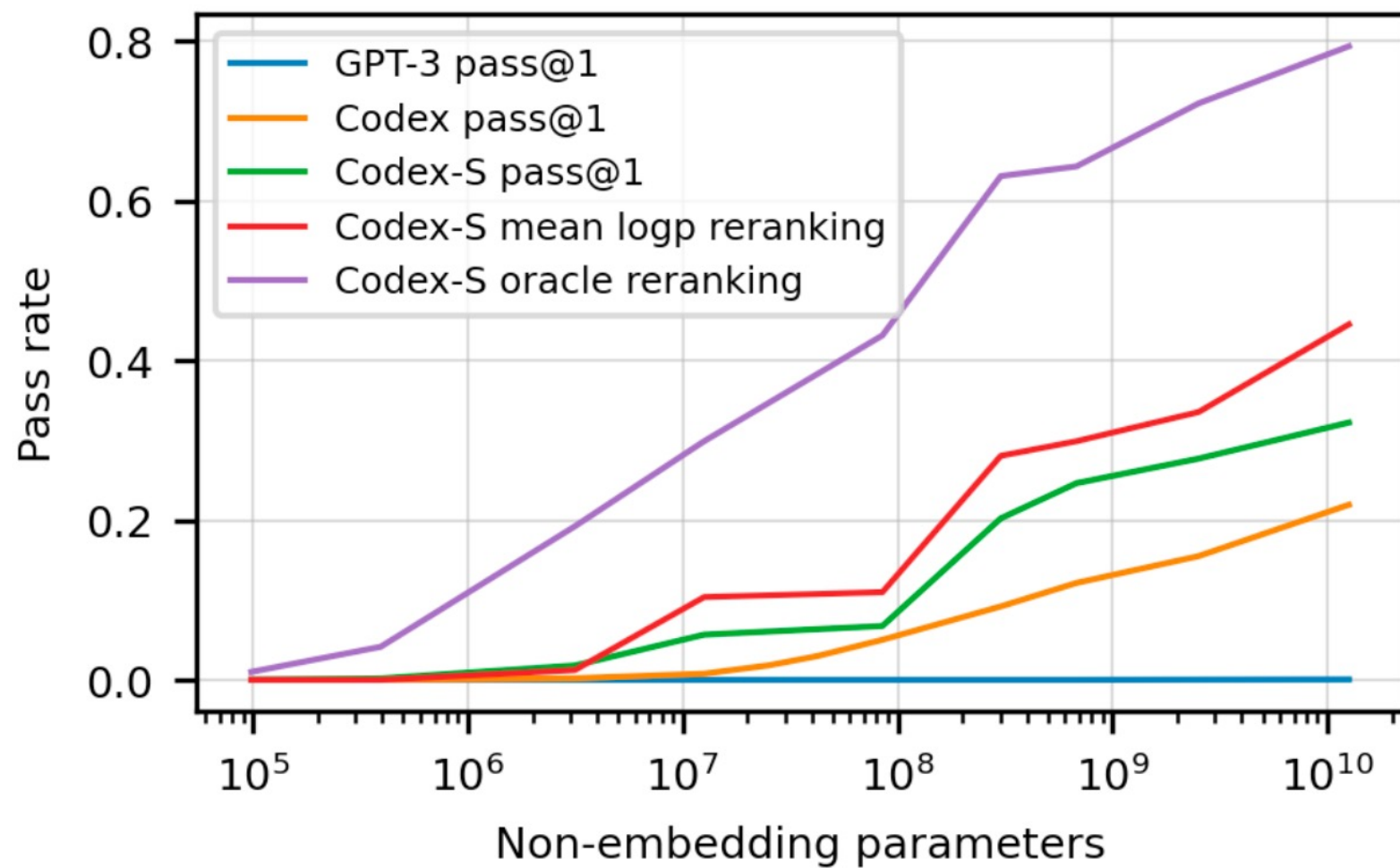


Efficiency and Flow*



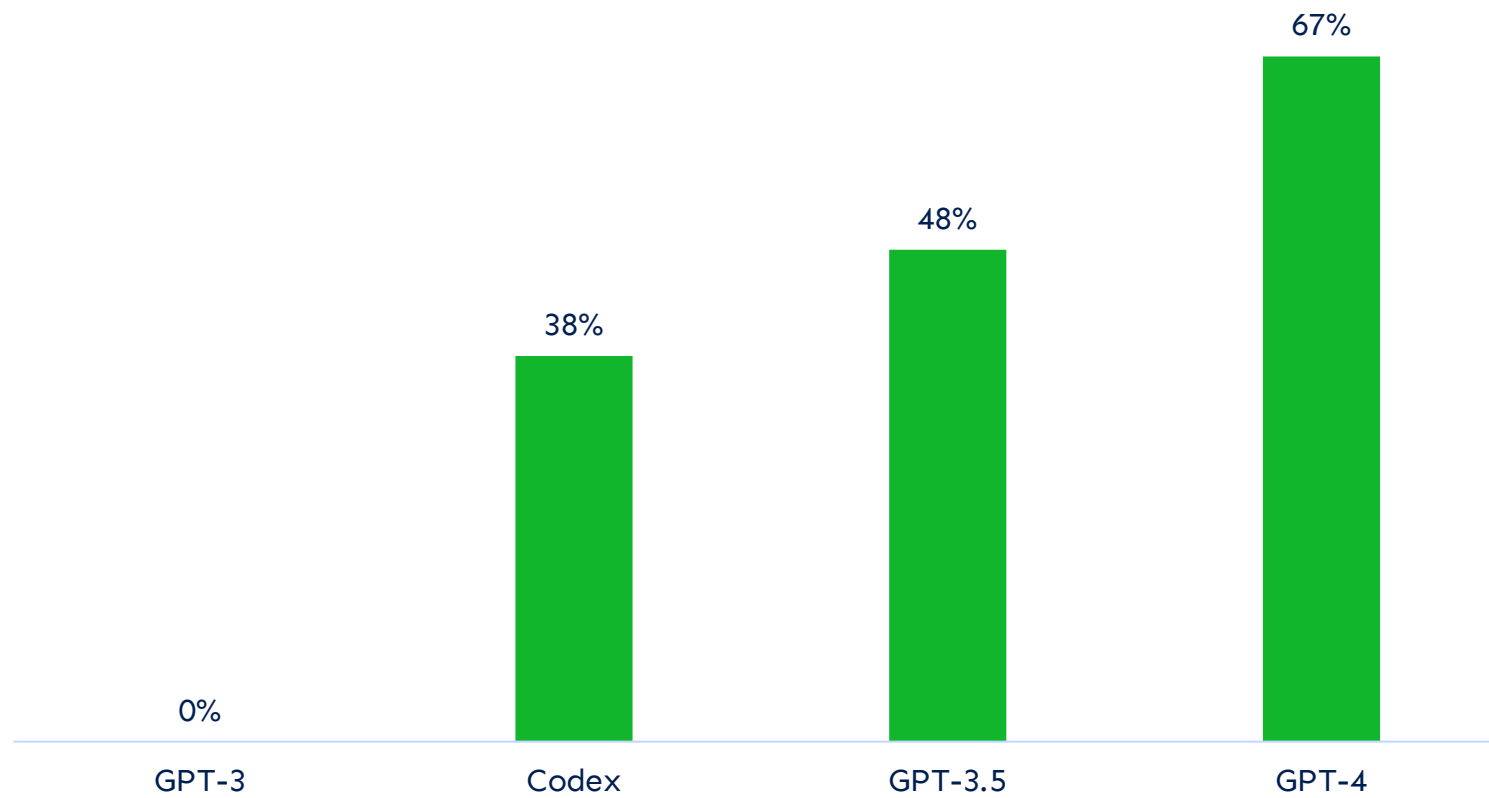


Codex and Codex-S Performance





HumanEval 0-shot performance





We recruited
 **95**
developers, and split them randomly into two groups.

We gave them the task of writing a web server in JavaScript

 **45 Used**
GitHub Copilot

 **78%**
finished

 **1 hour, 11 minutes**
average to complete the task



 **50 Did not use**
GitHub Copilot

 **70%**
finished

 **2 hours, 41 minutes**
average to complete the task



Results are statistically significant ($P=.0017$) and the 95% confidence interval is [21%, 89%]



What are the
problems with
Copilot?



Is Copilot a stochastic parrot?



Is Copilot a stochastic parrot?

453,780

Suggestions

41 (0.009%)

Recitations



Is Copilot a stochastic parrot?

Copilot recites snippets it has seen over 10 times in the training codebase.

Copilot recites when it has no context.



Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions

Publisher: IEEE

Cite This

PDF

Hammond Pearce ; Baleegh Ahmad ; Benjamin Tan ; Brendan Dolan-Gavitt ; Ramesh Karri [All Authors](#)

6

Paper
Citations

2226

Full
Text Views



Abstract

Document Sections

[I. Introduction](#)

[II. Background and Related Work](#)

[III. Using Github Copilot](#)

[IV. Experimental Method](#)

[V. Experimental Investigation of Github Copilot](#)

Show Full Outline ▾

[Authors](#)

[Figures](#)

[References](#)

Abstract:

There is burgeoning interest in designing AI-based systems to assist humans in designing computing systems, including tools that automatically generate computer code. The most notable of these comes in the form of the first self-described 'AI pair programmer', GitHub Copilot, which is a language model trained over open-source GitHub code. However, code often contains bugs—and so, given the vast quantity of unvetted code that Copilot has processed, it is certain that the language model will have learned from exploitable, buggy code. This raises concerns on the security of Copilot's code contributions. In this work, we systematically investigate the prevalence and conditions that can cause GitHub Copilot to recommend insecure code. To perform this analysis we prompt Copilot to generate code in scenarios relevant to high-risk cybersecurity weaknesses, e.g. those from MITRE's "Top 25" Common Weakness Enumeration (CWE) list. We explore Copilot's performance on three distinct code generation axes—examining how it performs given diversity of weaknesses, diversity of prompts, and diversity of domains. In total, we produce 89 different scenarios for Copilot to complete, producing 1,689 programs. Of these, we found approximately 40% to be vulnerable.

Published in: [2022 IEEE Symposium on Security and Privacy \(SP\)](#)

Date of Conference: 22-26 May 2022

INSPEC Accession Number: 21905589

Date Added to IEEE Xplore: 27 July 2022

DOI: [10.1109/SP46214.2022.9833571](#)



Does Copilot generate vulnerabilities?

1,689

Generated programs

39.33%

Top suggestions with vulnerability



“We hypothesize that the presence of vulnerable SQL in a codebase [...] has the strongest impact upon whether or not Copilot will itself generate SQL code vulnerable to injection ”



Do Users Write More Insecure Code with AI Assistants?

Neil Perry *
Stanford University

Megha Srivastava *
Stanford University

Deepak Kumar
Stanford University

Dan Boneh
Stanford University

[cs.CR] 16 Dec 2022

Abstract—We conduct the first large-scale user study examining how users interact with an AI Code assistant to solve a variety of security related tasks across different programming languages. Overall, we find that participants who had access to an AI assistant based on OpenAI’s `codex-davinci-002` model wrote significantly less secure code than those without access. Additionally, participants with access to an AI assistant were more likely to believe they wrote secure code than those without access to the AI assistant. Furthermore, we find that participants who trusted the AI less and engaged more with the language and format of their prompts (e.g. re-phrasing, adjusting temperature) provided code with fewer security vulnerabilities. Finally, in order to better inform the design of future AI-based Code assistants, we provide an in-depth analysis of participants’ language and interaction behavior, as well as release our user interface as an instrument to conduct similar studies in the future.

- **RQ1:** Does the distribution of security vulnerabilities users introduce differ based on usage of an AI assistant?
- **RQ2:** Do users trust AI assistants to write secure code?
- **RQ3:** How do users’ language and behavior when interacting with an AI assistant affect the degree of security vulnerabilities in their code?

We found that participants with access to an AI assistant often produced *more* security vulnerabilities than those without access, with particularly significant results for string encryption and SQL injection (Section 4). Surprisingly, we also found that participants provided access to an AI assistant were more likely to believe that they wrote secure code than those without access to the AI assistant (Section 5). Finally, we conducted an in-depth analysis of the different ways participants interacted with the AI assistant, such as including helper functions in their input prompt or adjusting model parameters, and found that those who trusted the AI



“We find that having access to the AI assistant results in more security vulnerabilities across multiple questions”



“For every question, participants having access to the AI assistant believed their answers were more secure, despite often providing more insecure answers.”



Evaluating Large Language Models Trained on Code

Mark Chen^{*1} Jerry Tworek^{*1} Heewoo Jun^{*1} Qiming Yuan^{*1} Henrique Ponde de Oliveira Pinto^{*1}
 Jared Kaplan^{*2} Harri Edwards¹ Yuri Burda¹ Nicholas Joseph² Greg Brockman¹ Alex Ray¹ Raul Puri¹
 Gretchen Krueger¹ Michael Petrov¹ Heidy Khlaaf³ Girish Sastry¹ Pamela Mishkin¹ Brooke Chan¹
 Scott Gray¹ Nick Ryder¹ Mikhail Pavlov¹ Alethea Power¹ Lukasz Kaiser¹ Mohammad Bavarian¹
 Clemens Winter¹ Philippe Tillet¹ Felipe Petroski Such¹ Dave Cummings¹ Matthias Plappert¹
 Fotios Chantzis¹ Elizabeth Barnes¹ Ariel Herbert-Voss¹ William Hebguss¹ Alex Nichol¹ Alex Paino¹
 Nikolas Tezak¹ Jie Tang¹ Igor Babuschkin¹ Suchir Balaji¹ Shantanu Jain¹ William Saunders¹
 Christopher Hesse¹ Andrew N. Carr¹ Jan Leike¹ Josh Achiam¹ Vedant Misra¹ Evan Morikawa¹
 Alec Radford¹ Matthew Knight¹ Miles Brundage¹ Mira Murati¹ Katie Mayer¹ Peter Welinder¹
 Bob McGrew¹ Dario Amodei² Sam McCandlish² Ilya Sutskever¹ Wojciech Zaremba¹

Abstract

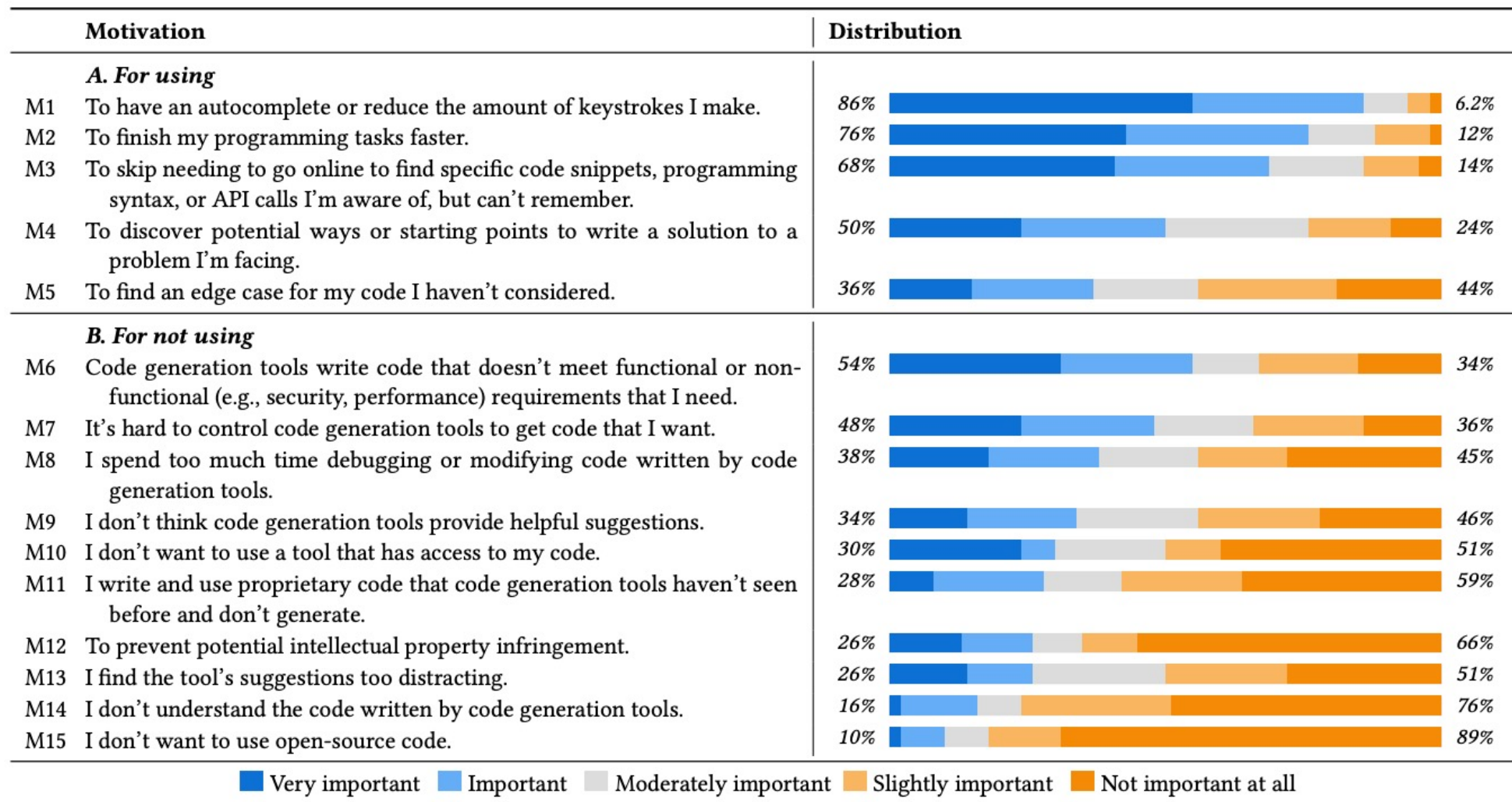
We introduce Codex, a GPT language model fine-tuned on publicly available code from GitHub, and study its Python code-writing capabilities. A distinct production version of Codex powers GitHub Copilot. On [HumanEval](#), a new evaluation set we release to measure functional correctness for synthesizing programs from docstrings, our model solves 28.8% of the problems, while GPT-3 solves 0% and GPT-J solves 11.4%. Furthermore, we find that repeated sampling from the model is a surprisingly effective strategy for producing working solutions to difficult prompts. Using this method, we solve 70.2% of our problems with 100 samples per problem. Careful investigation of our model reveals its limitations, including difficulty with docstrings describing long chains of operations and with binding operations to variables. Finally, we discuss the potential broader impacts of deploying powerful code generation technologies, covering safety, security, and economics.

1. Introduction

Scalable sequence prediction models ([Graves, 2014](#); [Vaswani et al., 2017](#); [Child et al., 2019](#)) have become a general-purpose method for generation and representation learning in many domains, including natural language processing ([Mikolov et al., 2013](#); [Sutskever et al., 2014](#); [Dai & Le, 2015](#); [Peters et al., 2018](#); [Radford et al., 2018](#); [Devlin et al., 2018](#)), computer vision ([Van Oord et al., 2016](#); [Menick & Kalchbrenner, 2018](#); [Chen et al., 2020](#); [Bao et al., 2021](#)), audio and speech processing ([Oord et al., 2016](#); [2018](#); [Dhariwal et al., 2020](#); [Baevski et al., 2020](#)), biology ([Alley et al., 2019](#); [Rives et al., 2021](#)), and even across multiple modalities ([Das et al., 2017](#); [Lu et al., 2019](#); [Ramesh et al., 2021](#); [Zellers et al., 2021](#)). More recently, language models have also fueled progress towards the longstanding challenge of program synthesis ([Simon, 1963](#); [Manna & Waldinger, 1971](#)), spurred by the presence of code in large datasets ([Husain et al., 2019](#); [Gao et al., 2020](#)) and the resulting programming capabilities of language models trained on these datasets ([Wang & Komatsuzaki, 2021](#)). Popular language modeling objectives like masked language modeling ([Devlin et al., 2018](#)) and span prediction ([Raffel et al., 2020](#)) have also been adapted to train their programming counterparts CodeBERT ([Feng et al., 2020](#)) and PyMT5 ([Clement et al.,](#)



“Codex will generate code that is as similar as possible to its training distribution. [...] If the user has some subtle mistakes in their code, Codex may *deliberately* suggest code that superficially appears good but is incorrect.”





How to use Copilot?





1. In education, use Copilot to **understand** code, not to cut corners.



2. Copilot does not replace proper testing.
It may facilitate it.



3. Copilot does not replace **code reviews**, in particular for junior developers.



4. Copilot can write code, but it is not proficient in software engineering or system design.



“I use Copilot every day. [...] It doesn't save me a lot of thinking, but it helps me take care of the boring stuff. ”

Guido Van Rossum, Inventor of Python



Ego Slide

Emanuele Fabbiani

Head of AI at xstream
PhD from UniPV

Speaker and track organizer at AMLD Lausanne,
ODSC, PyCon, eRUM, PyMI, Pisa.dev.

Course creator at UniPV, Almo Collegio Borromeo,
Treccani.

Lecturer in Italy, Poland, and Switzerland.

